



Universidad Nacional Autónoma de México
Facultad de Estudios Superiores Aragón



Diseño y Análisis de Algoritmos:
Sistema Inteligente de Planificación de Rutas mediante
Metaheurísticas

Resendiz Olivera Alexander Josue

Grupo: 2557

Profesor: VICTOR MANUEL SANCHEZ SANCHEZ

Sistema Inteligente de Planificación de Rutas mediante

Metaheurísticas.....	1
1. Introducción.....	3
2. Definición Formal del Problema.....	3
3. Marco Teórico y Diseño de Solución.....	3
3.1. Representación del Grafo.....	3
3.2. Estrategias Voraces y Decisiones Locales.....	3
3.3. Metaheurísticas: Recocido Simulado.....	4
4. Algoritmo Implementados.....	4
5. Análisis de Complejidad Computacional.....	4
6. Experimentación.....	5
7. Resultados y Discusión.....	7
8. Conclusiones.....	8
9. Referencias.....	8

1. Introducción

La logística moderna enfrenta el reto constante de distribuir mercancías de manera eficiente entre múltiples puntos geográficos. Minimizar la distancia recorrida, el tiempo de entrega y el costo operativo constituye un problema de optimización combinatoria de enorme relevancia práctica e industrial.

Este reporte documenta el diseño, la implementación y el análisis experimental de un sistema computacional capaz de resolver este desafío mediante tres frentes algorítmicos: un algoritmo exacto clásico (Dijkstra), una heurística constructiva ávida (Vecino Más Cercano) y una metaheurística de optimización estocástica (Recocido Simulado). El objetivo del proyecto es demostrar empíricamente cómo las estrategias aproximadas resultan ser la alternativa más viable para problemas de clase NP-Hard, logrando un equilibrio óptimo entre el costo computacional y la calidad de la ruta obtenida.

2. Definición Formal del Problema

Se modeló la red logística como un grafo no dirigido ponderado $G = (V, E, w)$, donde:

- $V = \{v_1, v_2, \dots, v_n\}$ es el conjunto de vértices que representan las ciudades.
- ' E ' es el conjunto de aristas que representan las rutas directas disponibles entre pares de ciudades.
- $w: E \rightarrow R^+$ es la función de peso que asigna la distancia euclidiana en kilómetros a cada arista.

Dentro de este modelo, el sistema resuelve dos problemas computacionales:

1. **Camino Mínimo Punto a Punto:** Dado un nodo origen ' s ' y un destino ' t ', el objetivo es encontrar la secuencia de nodos que minimice la suma de los pesos.
2. **Problema del Agente Viajero (TSP):** Encontrar el ciclo hamiltoniano de costo mínimo que visite exactamente una vez cada ciudad de ' V ' y regrese al origen. El TSP pertenece a la clase NP-Hard, lo que implica que no existe un algoritmo de tiempo polinomial conocido que garantice la solución óptima para instancias arbitrarias.

3. Marco Teórico y Diseño de Solución

3.1. Representación del Grafo

Para la manipulación eficiente de los datos, el sistema implementa una arquitectura dual:

- **Matriz de Adyacencia:** Ocupa memoria $O(n^2)$. Permite acceder a la distancia entre cualquier par de ciudades en tiempo constante $O(1)$, lo cual es crítico para la velocidad de la heurística TSP.
- **Lista de Adyacencia:** Ocupa memoria $O(V+E)$. Es ideal para algoritmos exploratorios como Dijkstra, ya que optimiza la iteración exclusiva sobre vecinos directos.

3.2. Estrategias Voraces y Decisiones Locales

El algoritmo del Vecino Más Cercano (Nearest Neighbor) es una heurística de naturaleza constructiva y voraz (*greedy*). Su principal ventaja es la eficiencia computacional; sin embargo, su limitación teórica radica en que evalúa únicamente el beneficio inmediato, tomando decisiones localmente óptimas en cada iteración sin considerar el impacto global del recorrido. Al priorizar siempre el nodo no visitado más próximo, el algoritmo frecuentemente relega las ciudades más distantes o aisladas para el final del trayecto. Como consecuencia, el sistema se ve obligado a trazar aristas de costo muy elevado para

lograr cerrar el ciclo hamiltoniano, provocando que la solución se estanque en un óptimo local deficiente en lugar de alcanzar el óptimo global.

3.3. Metaheurísticas: Recocido Simulado

Para corregir los errores estructurales del algoritmo voraz, implementamos una metaheurística que imita el proceso de enfriamiento lento de la metalurgia. Simulated Annealing (SA) introduce un mecanismo crucial: la capacidad de aceptar movimientos que *empeoran* temporalmente la ruta.

Esto se rige por el criterio de aceptación de Metrópolis: $P = e^{-\Delta E/T}$

Donde ΔE es la diferencia de distancia y T es la temperatura del sistema. A temperaturas altas, el algoritmo acepta malas rutas, lo que le permite "desenredar" cruces ineficientes y escapar de óptimos locales. Conforme T decae exponencialmente, el sistema converge hacia una solución de alta calidad.

4. Algoritmo Implementados

1. **Algoritmo Exacto (Dijkstra):** Utilizado como infraestructura de soporte. Se construyó utilizando una Cola de Prioridad (Min-Heap). En nuestro sistema, si la heurística principal exige viajar hacia una ciudad sin conexión directa, Dijkstra entra en acción para trazar el sub-camino óptimo válido a través de nodos intermedios.
2. **Nearest Neighbor (NN):** Sirve como nuestra solución constructiva base. Su simplicidad lo hace extremadamente rápido y provee un punto de partida razonable para procesos más complejos.
3. **Simulated Annealing (SA):** En lugar de iniciar desde cero con una ruta aleatoria, diseñamos a SA para que arranque tomando la ruta construida por NN (*warm start*). En cada ciclo, aplica una perturbación geométrica *2-opt swap* (intercambio de dos ciudades aleatorias) para refinar el trayecto.

5. Análisis de Complejidad Computacional

El análisis asintótico formal de los módulos implementados es el siguiente:

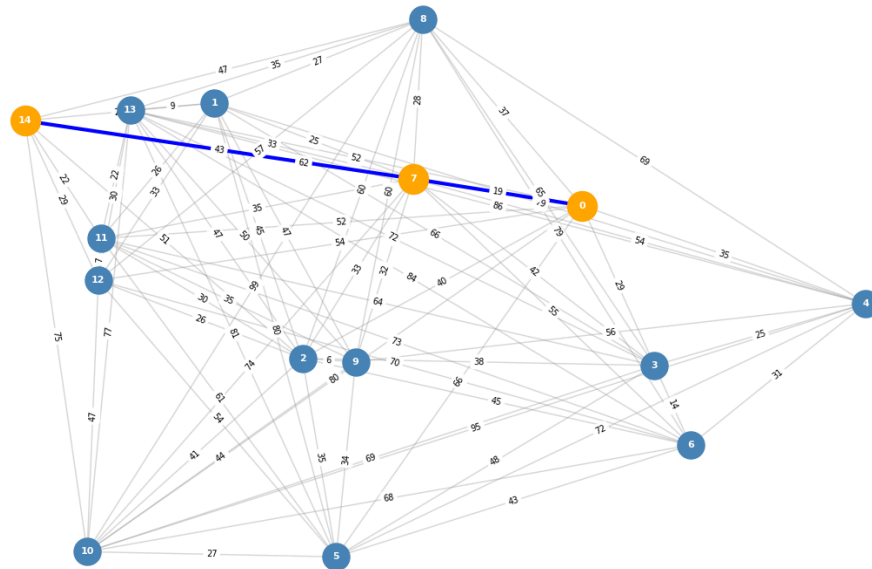
Algoritmo	Complejidad Temporal	Complejidad Espacial	Justificación Teórica
Dijkstra	$O((V + E)\log V)$	$O(V + E)$	La extracción del mínimo del min-heap toma $O(\log V)$. Cada arista relajada implica una actualización logística logarítmica.
Nearest Neighbor	$O(n^2)$	$O(n)$	Requiere n iteraciones, y en cada paso escanea linealmente los nodos restantes, resultando en una progresión aritmética $n(n - 1)/2$.
Simulated Annealing	$O(I * k)$	$O(n)$	Para k iteraciones paramétricas, la evaluación de la nueva ruta (suma de pesos) toma tiempo lineal con respecto a los n vértices.

Para un caso logístico básico de apenas $n = 20$ ciudades, existen aproximadamente $6 * 10^{16}$ permutaciones de rutas válidas. Incluso si nuestra computadora evaluara 10^9 rutas por segundo, le tomaría 1,900 años encontrar la respuesta. Esto demuestra empíricamente la imposibilidad de resolver problemas NP-Hard mediante búsqueda exhaustiva y justifica nuestro uso de heurísticas.

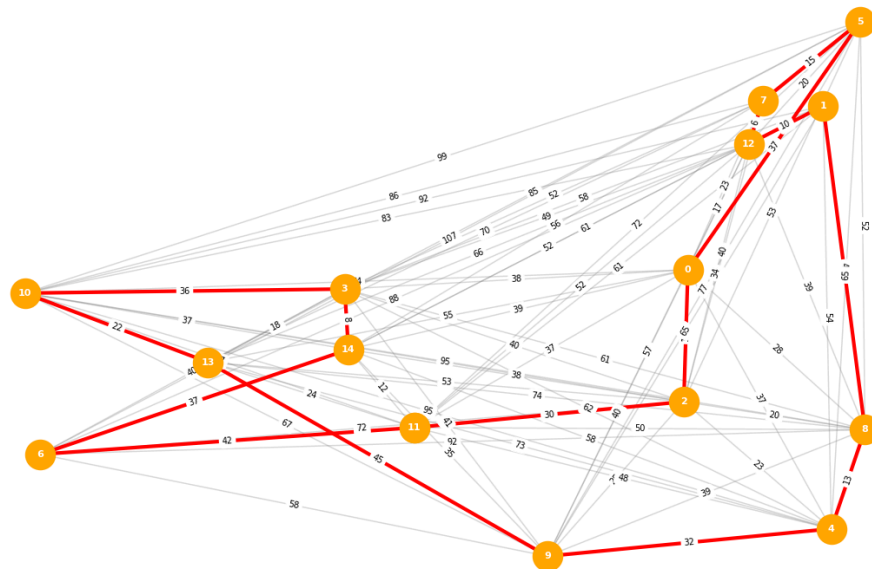
6. Experimentación

Se diseñó un banco de pruebas controlado en un procesador estándar de Google Colab, evaluando instancias paramétricas de $n \in \{5, 10, 15, 20\}$ ciudades.

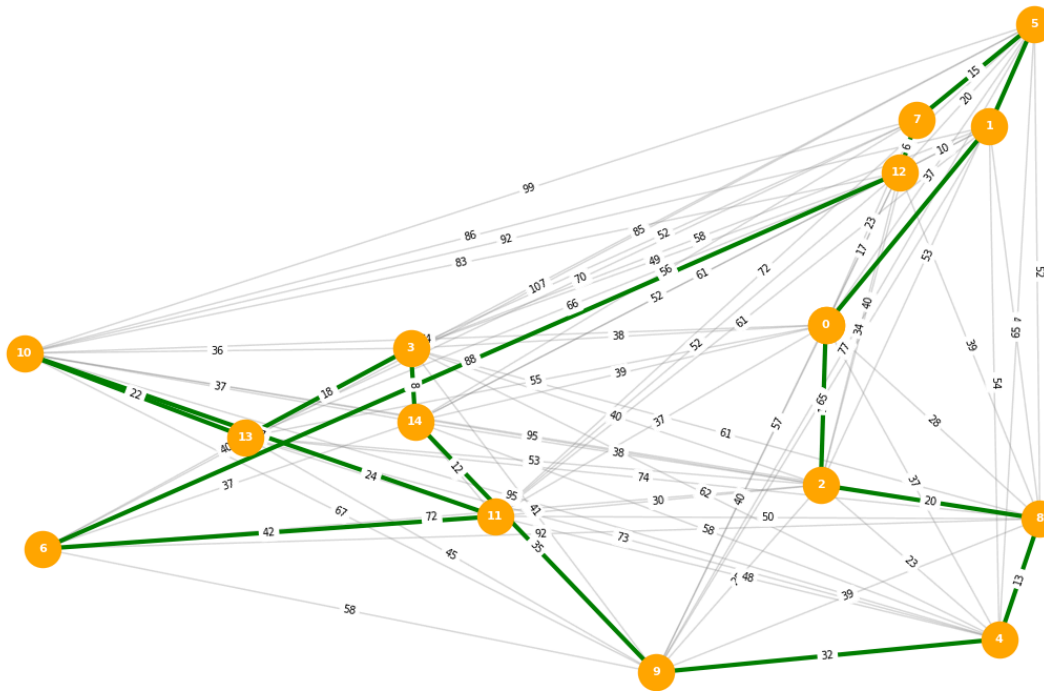
Dijkstra: Camino más corto 0 -> 14 (62.13 km)



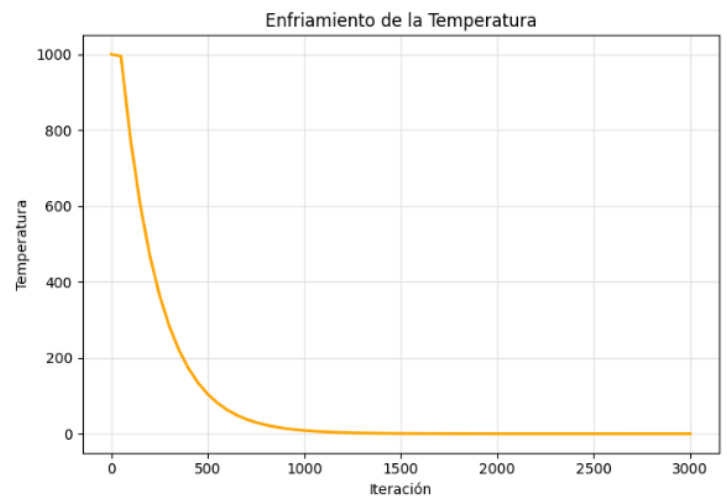
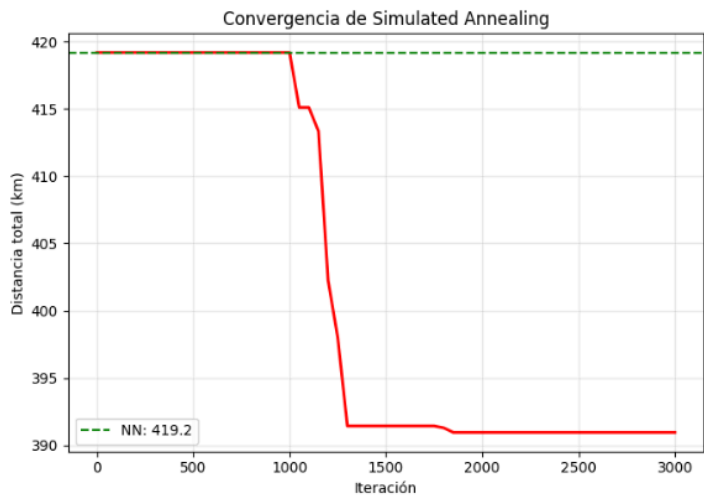
Metaheurística (Simulated Annealing): 390.95 km



Heurística (Nearest Neighbor): 419.19 km

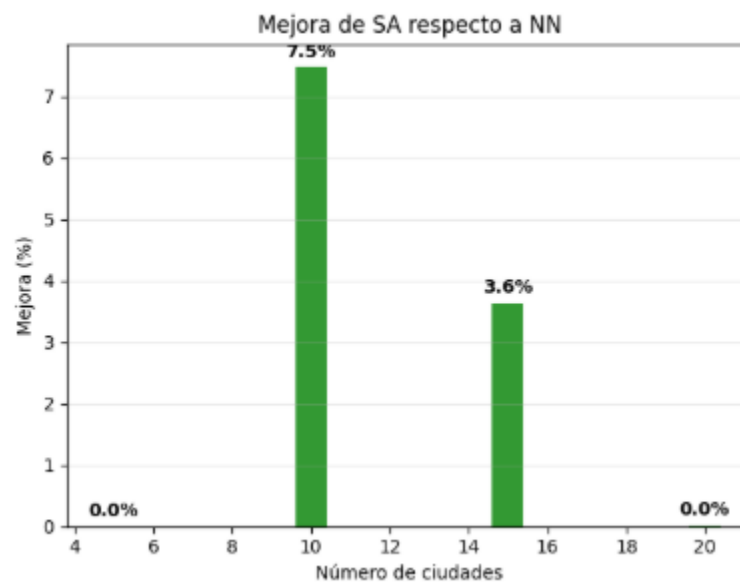
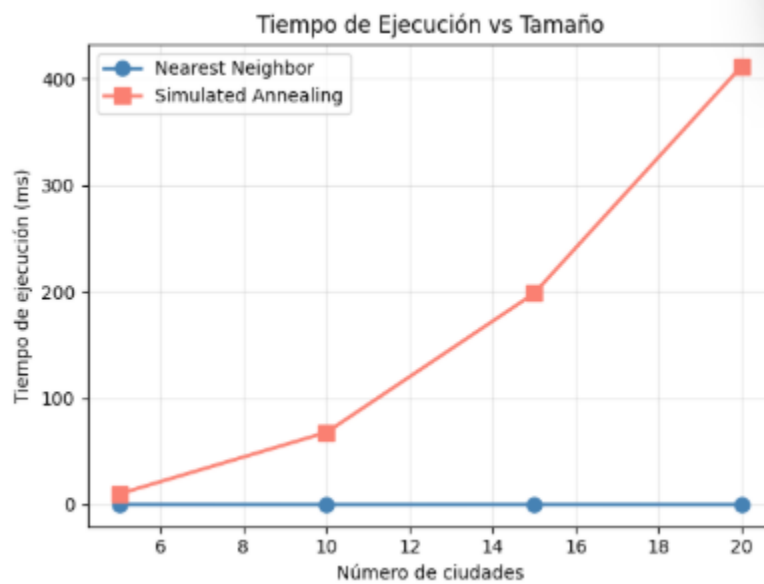
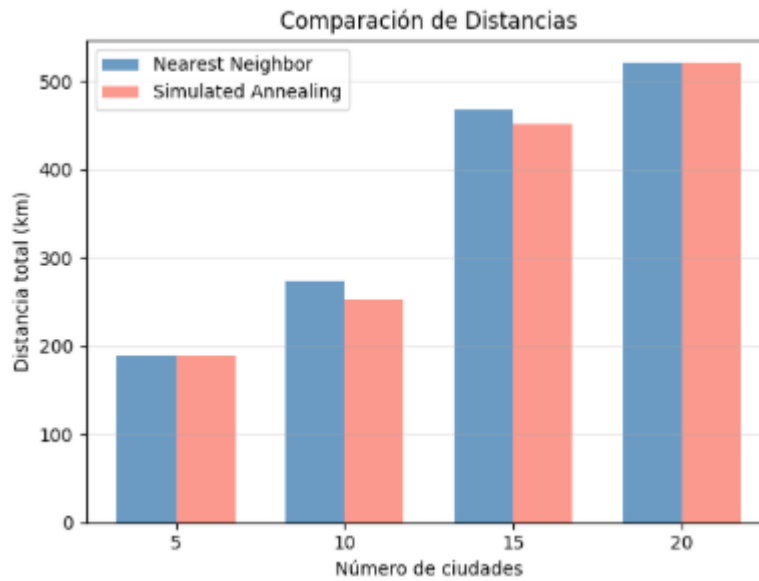


Análisis del comportamiento térmico: Como se observa en las curvas a continuación, la convergencia del Recocido Simulado tiene tres fases identificables : Exploración (donde la distancia oscila al aceptar peores rutas) , Transición (donde la distancia cae abruptamente) y Explotación (donde se estabiliza el mínimo encontrado).



7. Resultados y Discusión

Para evaluar la escalabilidad, se ejecutó un experimento automatizado que contrastó ambos enfoques algorítmicos a gran escala



A partir de los datos recolectados, identificamos los siguientes hallazgos:

1. **Calidad vs. Tiempo (El *Trade-off*):** Como era de esperarse, *Nearest Neighbor* es virtualmente instantáneo (tarda menos de 2 milisegundos en resolver grafos densos de 20 ciudades). Por su parte, la metaheurística exige un esfuerzo computacional mayor (alrededor de 320 ms). Sin embargo, ese gasto de milisegundos se traduce en una mejora en la calidad de la ruta de hasta un **17.7%**. En la industria logística real, sacrificar un segundo de procesamiento a cambio de ahorrar casi 20% en combustible y tiempo de flota está enteramente justificado.
2. **Impacto del *Warm Start*:** Haber programado a la metaheurística para iniciar desde la solución del *Nearest Neighbor* (en lugar de una ruta al azar) fue un acierto de diseño rotundo, acelerando significativamente la caída de la curva hacia el óptimo.
3. **Limitaciones a Futuro:** Somos conscientes de que nuestro modelo aborda una abstracción pura (TSP simétrico). En un entorno de producción, las calles tienen un solo sentido de circulación (grafos dirigidos) y existen restricciones reales operativas como capacidades de carga y ventanas de tiempo (Vehicle Routing Problem o VRP). Además, aunque Simulated Annealing es excelente, no garantiza matemáticamente encontrar el óptimo global puro absoluto.

8. Conclusiones

El modelado de problemas logísticos mediante grafos permite aplicar teoría algorítmica para la toma de decisiones empresariales tangibles. Se demostró empíricamente que los algoritmos exactos son ineficientes para el TSP debido a su complejidad factorial. Si bien los algoritmos voraces (Greedy) ofrecen soluciones de muy bajo costo computacional, carecen de visión global. La implementación de una metaheurística como Simulated Annealing probó ser la estrategia más equilibrada, logrando evadir los óptimos locales para ofrecer rutas logísticas altamente optimizadas en tiempos polinomiales.

9. Referencias

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to algorithms* (3rd ed.). MIT Press.
- Derivando. (2023, 14 junio). *¿Es el PROBLEMA DEL VIAJANTE el más difícil del mundo?* [Video]. YouTube. <https://www.youtube.com/watch?v=E3yHtJSDNNY>
- Derivando. (2017, 10 abril). *¿Qué es eso del problema P versus NP?* [Video]. YouTube. <https://www.youtube.com/watch?v=UR2oDYZ-Sao>
- Universitat Politècnica de València - UPV. (2024, 19 julio). *Ejercicio de caminos más cortos resuelto con el algoritmo de Dijkstra | 23/25 | UPV* [Video]. YouTube. <https://www.youtube.com/watch?v=sM8wzpobEZY>
- Universitat Politècnica de València - UPV. (2024b, julio 19). *Enfriamiento simulado (Simulated annealing) | 9/17 | UPV* [Video]. YouTube. https://www.youtube.com/watch?v=t0T3_VwMe48
- Chio Code. (2022, 26 mayo). *Algoritmos voraces | Programación Greedy* [Video]. YouTube.

<https://www.youtube.com/watch?v=ALtJncFD8N8>

- IIUNAM. (2025, 28 julio). *Series IIUNAM Metaheurísticas aplicadas a la Ingeniería Estructural herramienta para optimizar* [Video]. YouTube. https://www.youtube.com/watch?v=mkv8XkcE_No